## GRAPH ALGORITHMS

```
class DSU
{
   int n;
   vector<int> repArr;
   vector<int> rank;

public:
   DSU(int n) : n(n), repArr(n), rank(n, 1)
   {
      for (int i = 0; i < n; i++)
      {
         repArr[i] = i;
      }
   }
   int find(int a)
   {
      if (repArr[a] == a)
      {
         return a;
      }
      else
      {
         return repArr[a] = find(repArr[a]);
      }
   }
   void unite(int a, int b)
   {
      a = find(a);
      b = find(b);

      if (a == b)
         return;

      if (rank[a] > rank[b])
      {
         repArr[b] = a;
      }
      else if (rank[a] < rank[b])
      {
         repArr[a] = b;
      }
      else
      {
         repArr[b] = a;
         rank[a]++;
      }
   }
   auto getArray()
   {
      return repArr;
   }
};

class Graph
{
   int V;
   vector<vector<pair<int, ll>>> aList;

   // Floyd Warshall Data
   vector<vector<ll>> dist;
   vector<vector<int>> next;

public:
   Graph(const vector<vector<pair<int, ll>>>& aList) :
V(aList.size()), aList(aList)
   {

   }

   // Path Finding Algorithms
   auto bellmanFord(int src, int dest)
   {
      vector<ll> cost(V, INF);
      vector<int> parent(V, -1);
      cost[src] = 0;

      for (int it = 0; it < V - 1; it++)
      {
         for (int currVer = 0; currVer < V; currVer++)
         {
            for (auto [nextVer, nextCost] : aList[currVer])
            {
               if (cost[currVer] != INF && cost[currVer] +
nextCost < cost[nextVer])
               {
                  cost[nextVer] = cost[currVer] + nextCost;
                  parent[nextVer] = currVer;
               }
            }
         }
      }

      for (int currVer = 0; currVer < V; currVer++)
      {
         for (auto [nextVer, nextCost] : aList[currVer])
         {
            if (cost[currVer] != INF && cost[currVer] +
nextCost < cost[nextVer])
            {
               cout << "NEGATIVE CYCLE EXISTS,
CANNOT SOLVE\n";
               return pair<ll, vector<int>> {-1, {-1}};
            }
         }
      }

      if (cost[dest] == INF)
      {
         cout << "DESTINATION UNREACHABLE
FROM SOURCE\n";
         return pair<ll, vector<int>> {INF, {-1}};
      }
      vector<int> path;
      int currVer = dest;
      while (currVer != -1)
      {
         path.push_back(currVer);
         currVer = parent[currVer];
      }
      reverse(path.begin(), path.end());
      return pair<ll, vector<int>> {cost[dest], path};
   }
   auto floydWarshall()
   {

      dist.assign(V, vector<ll>(V, INF));
      next.assign(V, vector<int>(V, -1));

      for (int currVer = 0; currVer < V; currVer++)
      {
         for (auto [nextVer, nextCost] : aList[currVer])
         {
            dist[currVer][nextVer] = nextCost,
            next[currVer][nextVer] = nextVer;
         }
      }

      for (int i = 0; i < V; i++)
      {
         dist[i][i] = 0;
         next[i][i] = i;
      }

      for (int k = 0; k < V; k++)
      {
         for (int i = 0; i < V; i++)
         {
            for (int j = 0; j < V; j++)
            {
               if (dist[k][j] != INF && dist[i][k] != INF &&
                  dist[i][j] > dist[k][j] + dist[i][k])
               {
                  dist[i][j] = dist[k][j] + dist[i][k];
                  next[i][j] = next[i][k];
               }
            }
         }
      }
   }
   auto floydWarshallPath(int src, int dest)
   {
      if (next[src][dest] == -1)
      {
         cout << "DESTINATION UNREACHABLE
FROM SOURCE\n";
         return vector<int>{-1};
      }

      vector<int> path;
      int curr = src;
      while (curr != dest)
      {
         path.push_back(curr);
         curr = next[curr][dest];
      }
      path.push_back(dest);
      return path;
   }
   ll floydWarshallCost(int src, int dest)
   {
      return dist[src][dest];
   }
   // MST Algorithms
   auto prim()
   {
      vector<bool> visited(V, false);
      vector<int> parent(V, -1);
      vector<ll> cost(V, INF);
      priority_queue
      <
         pair<ll, int>,
         vector<pair<ll, int>>,
         greater<pair<ll, int>>
      > pq;

      cost[0] = 0;
      pq.push({0, 0});

      while (!pq.empty())
      {
         auto [currCost, currVer] = pq.top();
         pq.pop();

         if (visited[currVer])
            continue;

         visited[currVer] = true;
         for (auto [nextVer, nextCost] : aList[currVer])
         {
            if (!visited[nextVer] && cost[nextVer] >
nextCost)
            {
               cost[nextVer] = nextCost;
               pq.push({nextCost, nextVer});
               parent[nextVer] = currVer;
            }
         }
      }
      return parent;
   }
   static auto sortEdges(const pair<ll, pair<int, int>>& e1,
const pair<ll, pair<int, int>>& e2)
   {
      return e1.first < e2.first;
   }
   auto kruskal()
   {
      vector<pair<ll, pair<int, int>>> eList;
      for (int currVer = 0; currVer < V; currVer++)
      {
         for (auto [nextVer, nextCost] : aList[currVer])
         {
            eList.push_back({nextCost, {currVer,
nextVer}});
         }
      }

      sort(eList.begin(), eList.end(), sortEdges);

      vector<pair<int, int>> mstEdges;
      ll mstCost = 0;
      DSU dsu(V);
      for (auto [cost, edge] : eList)
      {
         auto [v1, v2] = edge;
         if (dsu.find(v1) != dsu.find(v2))
         {
            mstCost += cost;
            mstEdges.push_back({v1, v2});
            dsu.unite(v1, v2);
         }
      }

      if (mstEdges.size() != V - 1)
      {
         cout << "DISCONNECTED GRAPH, MST DOES
NOT EXIST\n";
         return pair<ll, vector<pair<int, int>>> {-1, {}};
      }

      return pair<ll, vector<pair<int, int>>> {mstCost,
mstEdges};
   }

};
```

## SORTING ALGORITHMS

### RADIX SORT USING COUNTERS

```
#include <bits/stdc++.h>
using namespace std;
using vint = vector<int>;

void rsort(vint& arr, int exp)
{
   vint count(10, 0);
   for (int& val: arr)
   {
      int dig = (val / exp) % 10;
      count[dig]++;
   }
   for (int i = 1; i < 10; i++)
   {
      count[i] += count[i-1];
   }

   vint output(arr.size());
   for (int i = arr.size() - 1; i >= 0; i--)
   {
      int dig = (arr[i] / exp) % 10;
      output[--count[dig]] = arr[i];
   }
   arr.swap(output);
}

auto radixSort(vint& arr)
{
   const int min = *min_element(arr.begin(), arr.end());
   if (min < 0)
   {
      for (int& num: arr)
      {
         num += abs(min);
      }
   }

   const int max = *max_element(arr.begin(), arr.end());
   for (int exp = 1; max / exp > 0; exp *= 10)
   {
      rsort(arr, exp);
   }
   if (min < 0)
   {
      for (int& num: arr)
      {
         num -= abs(min);
      }
   }
}
```

### MERGE SORT

```
#include <bits/stdc++.h>
using namespace std;

using vint = vector<int>;

void merge(vint& arr, int left, int mid, int right)
{
   int n1 = mid - left + 1;
   int n2 = right - mid;

   vint leftArr(n1);
   vint rightArr(n2);

   for (int i = 0; i < n1; i++)
   {
      leftArr[i] = arr[left + i];
   }

   for (int i = 0; i < n2; i++)
   {
      rightArr[i] = arr[mid + 1 + i];
   }
   int i = 0;
   int j = 0;
   int k = left;

   while (i < n1 && j < n2)
   {
      if (leftArr[i] <= rightArr[j])
      {
         arr[k] = leftArr[i];
         i++;
      }
      else
      {
         arr[k] = rightArr[j];
         j++;
      }

      k++;
   }

   while (i < n1)
   {
      arr[k] = leftArr[i];
      i++;
      k++;
   }

   while (j < n2)
   {
      arr[k] = rightArr[j];
      j++;
      k++;
   }
}

void mergeSort(vint& arr, int left, int right)
{
   if (left >= right)
   {
      return;
   }

   int mid = left + (right - left) / 2;

   mergeSort(arr, left, mid);
   mergeSort(arr, mid + 1, right);

   merge(arr, left, mid, right);
}
```

## DYNAMIC PROGRAMMING

### COIN CHANGE UNBOUNDED

```
coinchange_limited(const int requiredSum, const
vector<int>& coins)
{
   int n = coins.size();
   vector<vector<ll>> dp(n + 1, vector<ll>(requiredSum +
1));
   for (int col = 0; col <= requiredSum; col++)
   {
      dp[0][col] = 0;
   }
   dp[0][0] = 1;

   for (int row = 1; row < n + 1; row++)
   {
      int coin = coins[row - 1];
      for (int col = 0; col < requiredSum + 1; col++)
      {
         if (coin > col)
         {
            dp[row][col] = dp[row-1][col];
         }
         else
         {
            dp[row][col] = dp[row-1][col] + dp[row][col -
coin];
         }
      }
   }

   return dp.back()[requiredSum];
}
```

### COIN CHANGE MIN COINS

```
int minCoins2D(int requiredSum, const vector<int>&
coins)
{
   int n = coins.size();

   vector<vector<int>> dp(n + 1, vector<int>(requiredSum
+ 1, INF));

   // Base case: 0 coins needed to make sum 0
   for (int i = 0; i <= n; i++)
   {
      dp[i][0] = 0;
   }

   for (int row = 1; row <= n; row++)
   {
      int coin = coins[row - 1];

      for (int sum = 1; sum <= requiredSum; sum++)
      {
         if (coin > sum)
         {
            // Cannot use current coin
            dp[row][sum] = dp[row - 1][sum];
         }
         else
         {
            dp[row][sum] = min(dp[row - 1][sum], 1 +
dp[row][sum - coin]);
         }
      }
   }
```

**COIN CHANGE UNBOUNDED (contd.)**

```
if (dp[n][requiredSum] == INF)
        return -1;

    return dp[n][requiredSum];
}
```

**COIN CHANGE K COINS**

```
ll change_kCoins(int requiredSum, const vector<int>&
coins, int K)
{
    vector<vector<ll>> dp(requiredSum + 1, vector<ll>(K +
1, 0));

    for (int coin : coins)
    {
        for (int sum = coin; sum <= requiredSum; sum++)
        {
            for (int used = 1; used <= K; used++)
            {
                dp[sum][used] += dp[sum - coin][used - 1];
            }
        }
    }

    ll answer = 0;

    for (int used = 0; used <= K; used++)
    {
        answer += dp[requiredSum][used];
    }

    return answer;
}
```

**KNAPSACK**

```
int knapsack(const v_int& weights, const v_int& values,
int capacity)
{
    int n = weights.size();
    v2_int dp(n+1, v_int(capacity+1));

    for (int row = 1; row <= n; row++)
    {
        int weight = weights[row-1];
        int value  = values[row-1];
        for (int col = 1; col <= capacity; col++)
        {
            if (weight > col)
            {
                dp[row][col] = dp[row-1][col];
            }
            else
            {
                dp[row][col] = max(dp[row-1][col], dp[row-1]
[col-weight] + value);
            }
        }
    }

    return dp.back().back();
}
```

**UNBOUNDED KNAPSACK**

```
using ll = long long;
using v_int = vector<int>;
using v2_int = vector<vector<int>>;

int maxValue(const v_int& weights,const v_int& values,
int capacity)
{
    int n = weights.size();

    v2_int dp(n + 1, v_int(capacity + 1, 0));

    for (int row = 1; row <= n; row++)
    {
        int weight = weights[row - 1];
        int value = values[row - 1];

        for (int col = 1; col <= capacity; col++)
        {
            if (weight > col)
            {
                dp[row][col] = dp[row - 1][col];
            }
            else
            {
                dp[row][col] = max(
                    dp[row - 1][col],
                    dp[row][col - weight] + value
                );
            }
        }
    }

    return dp.back().back();
}
```

**MIN WHITESPACE**

```
using vll = vector<long long>;
using vll2 = vector<vector<long long>>;
using vstr = vector<string>;

auto minWhitespace(vstr& words, int capacity)
{
    int n = words.size();
    vll2 dp(n+1, vll(capacity+1, 0));

    for (int row = 1; row <= n; row++)
    {
        int wordSize = words[row-1].size();
        for (int col = 1; col <= capacity; col++)
        {
            if (wordSize > col)
            {
                dp[row][col] = dp[row-1][col];
            }
            else
            {
                dp[row][col] = max(dp[row-1][col], dp[row-1]
[col - wordSize] + wordSize);
            }
        }
    }
    return capacity - dp.back().back();
}
```

**FROG JUMP**

```
int frogJump(const vint& heights)
{
    int n = heights.size();

    // dp[i] = minimum cost required to reach stone i
    vint dp(n, 0);

    // Cost to reach stone 0 is 0 because the frog starts there.
    dp[0] = 0;

    // Stone 1 can only be reached from stone 0.
    if (n > 1)
    {
        dp[1] = abs(heights[1] - heights[0]);
    }

    for (int i = 2; i < n; i++)
    {
        // Jump from stone i - 1 to stone i.
        int oneJump =
            dp[i - 1]
            + abs(heights[i] - heights[i - 1]);

        // Jump from stone i - 2 to stone i.
        int twoJumps =
            dp[i - 2]
            + abs(heights[i] - heights[i - 2]);

        dp[i] = min(oneJump, twoJumps);
    }

    return dp[n - 1];
}
```